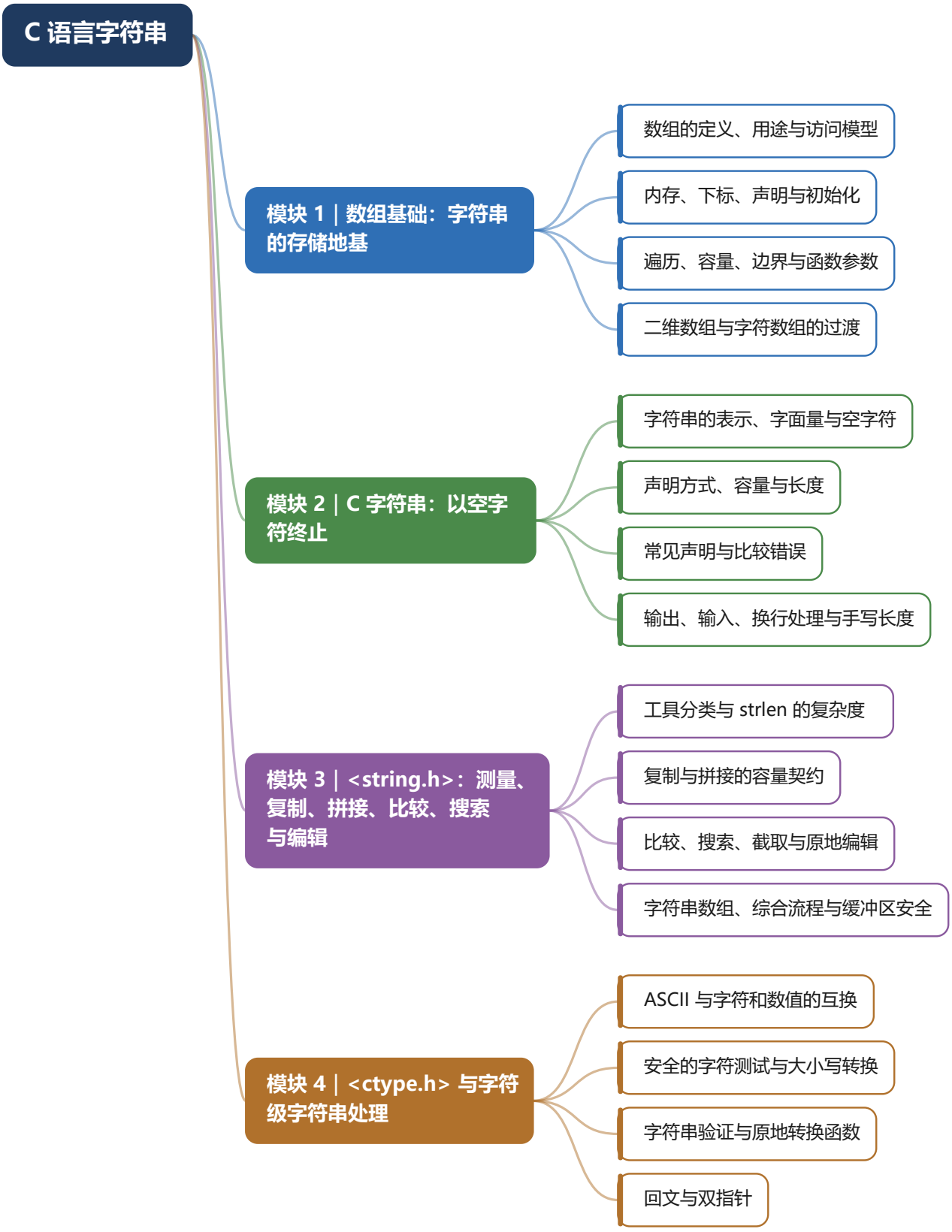
CST204 | C 语言字符串（Strings in C）复习笔记

> 来源：`CST204 - Strings in C.pdf`，共 58 页。本文按课件顺序完整重组；页码均指原 PDF 页码。术语第一次出现时保留英文，代码以 C17 常见语义解读。



C 语言字符串知识结构图：数组基础、字符串表示、string.h 工具箱和 ctype.h 字符处理。

模块 1｜数组基础：字符串的存储地基

1.1 数组的定义、用途与访问模型

> **A named collection of elements stored one after another.** (p9)
>
> 译：**数组（array）**是按顺序连续存放、拥有同一名称的一组元素。(p9)

- **数组的同类型性**：同一数组的每个元素具有相同数据类型，例如全为 `int`、`float` 或 `char`。（p9）
- **连续存储（contiguous）**：元素在内存中紧挨着排列；这使得可按下标直接定位。（p9）
- **固定容量**：数组大小在声明时确定，之后不能自动增长。（p9）
- **零基下标**：长度为 `n` 的数组合法下标是 `0` 到 `n - 1`。（p9）
- 用数组代替大量独立变量，循环即可统一读入、累计和处理；若大小会变化，应用一个命名常量而不是在多处重复字面量。（p8）
- 数组也可实现 ADT；例如同一个 List ADT 可由数组或链表实现。字符串则是以 `char` 数组表示的一种约定。（p5）
- 顺序查找可用于任意顺序数据，最坏为 `O(n)`；二分查找要求已排序，最坏次数为 `floor(log₂ n) + 1`（`n ≥ 1`）。（p6）

1.2 内存、下标、声明与初始化

- **地址公式**：`a[i]` 的地址为 `base address + i × sizeof(element type)`；故任意已知下标的访问是 `O(1)`。（p10）
- 下标可为任意整数表达式。`temps[m + 1]` 先计算下标再取元素；`temps[m] + 1` 则先取元素值再做加法，二者不能混淆。（p12）
- 声明语法是 `type name[size];`，如 `int x[5];`。（p11）
- `int x[5] = {40, 55, 63, 17, 22};` 给出了全部元素；`int y[] = {1, 2, 3};` 由编译器推断大小为 3。（p11）
- 部分初始化会把剩余元素置零：`int z[5] = {1, 2};` 的后三个元素为 `0`。（p11）
- 局部、非静态的未初始化数组元素值不确定，不能读取。（p11）

1.3 遍历、容量、边界与函数参数

- 循环变量自然适合作为数组下标；正向或反向遍历都合法，前提是始终落在 `0 … n - 1`。（p14）
- 求最大值的通用扫描模式：先令 `max = a[0]`，再从 `i = 1` 开始；仅当 `a[i] > max` 时更新。这避免重复比较第一个元素。（p15）
- **元素个数宏**：`#define LEN(arr) (sizeof(arr) / sizeof((arr)[0]))` 只适用于当前作用域内真正的数组对象。（p16）
- 数组作为函数参数会退化为指针，`sizeof` 不能再得出元素总数；函数必须另收一个长度参数。（p16–p18）

- C 不会自动检查数组边界。长度为 10 的数组写 `a[10]` 即越界，可能造成错误数据、崩溃或安全漏洞。（p17）
- 读入长度与循环前要验证：输入成功，且 `0 <= n && n <= capacity`；循环写为 `i < n`，不能误写成 `i <= n`。（p17）
- `sum(int a[], int n)` 实际收到的是首元素地址，不是数组副本；因而可读写调用方数组，并且必须依赖 `n` 控制范围。（p18）

1.4 二维数组与字符数组的过渡

- 二维数组是“数组的数组”，访问形式为 `A[row][col]`；内存按行优先（row-major）存放。（p19）
- `int B[2][3]` 应以双层循环逐行、逐列处理。（p19）
- 普通 `char word[5] = {'H','e','l','l','o'};` 只有字符数据，本身没有长度或结束位置；若要当作字符串，要么额外传长度，要么以特殊结束标记终止。（p20）

模块 2 | C 字符串：以 `\0` 结束的字符数组

2.1 字符串的表示、字面量与空字符

> **A string is a char array ending with ‘\0’.** (p22)
>
> 译：**C 字符串（string）**是以 `\0` 结尾的 `char` 数组。(p22)

- C 没有内建的 `string` 关键字；文本存放在 `char` 数组中。（p22）
- **空字符（null character）** `\0` 的数值为 `0`，是字符串的停止标志；它不同于字符 `'0'`，后者的 ASCII 值为 `48`。（p22）
- 例如 `"Hello"` 有 5 个可见字符，但需额外 1 个 `\0`，共占 6 个字节。（p22）
- ASCII 为字符分配数值，如 `'A' = 65`、`'a' = 97`、`'0' = 48`、`'\0' = 0`。（p22）
- `'A'` 是字符常量，类型是 `int`，`sizeof('A')` 为 `sizeof(int)`；`"A"` 是含 `'A'` 与隐藏 `\0` 的 2 字节字符串；`""` 是仅含 `\0` 的 1 字节空串。（p23）
- `char c = 'A';` 创建一个一字节字符对象；不能用 `char c = "A";`。（p23）

2.2 声明方式、容量与长度

- 手写初始化列表时必须自己补 `\0`：`char s1[6] = {'H','e','l','l','o','\0'};`。（p24）
- `char s2[] = "Hello";` 让编译器计算容量，结果为 6。（p24）
- `char s3[10] = "Hello";` 有 10 字节容量；字符串后面的剩余位置被零初始化。（p24）
- `const char *s4 = "Hello";` 指向只读字符串字面量；不能通过该指针修改内容。（p24）
- **容量（size/capacity）**由 `sizeof` 给出，是整个数组占用字节数，包含 `\0`；`sizeof` 不扫描字符串。（p25）

- **长度（length）** 由 `<string.h>` 的 `strlen` 给出，是首个 `\0` 前的字节数，不包含 `\0`，并要求传入有效的、已终止的字符串。（p25）
- 对 `char s[10] = "Hello";`，`sizeof s` 为 10，`strlen(s)` 为 5；`size_t` 值用 `%zu` 输出。（p25）
- 对指针使用 `sizeof` 得到的是指针本身大小，不是它所指字符串的容量。（p25）
- `strlen` 数的是字节，不保证等于用户看到的字符个数，例如多字节编码文本。（p25）

2.3 常见声明与比较错误

- `char s[5] = "Hello";` 没有给 `\0` 留空间，不能形成有效字符串；至少需要 6。（p26）
- 数组不可整体赋值：`s = "World";` 是编译错误；应复制内容，例如 `strcpy(s, "World")`，前提是容量足够。（p26）
- `s == "Hello"` 比较的是地址，不是文本；文本相等要写 `strcmp(s, "Hello") == 0`。（p26）
- 修改字符串字面量，如 `char *p = "Hi"; p[0] = 'h';`，是未定义行为。需要可修改副本时，用 `char p[] = "Hi";`。（p26）
- 有效字符串的核心不只是“字符数够”，而是可写存储空间中存在终止的 `\0`。（p27–p28）

2.4 输出、输入、换行处理与手写长度

- `printf("%s\n", s)` 按顺序输出，直到首个 `\0`；`puts(s)` 也输出字符串，但会额外添加换行。（p29）
- 没有 `\0` 时，`%s` 会越界继续读，属于未定义行为，可能打印垃圾或崩溃。（p29）
- `scanf("%19s", name)` 最多读 19 字节并补 `\0`，适用于单词；它在空格、制表符或换行处停止，返回值为 1 才表示成功读到一个字符串。（p30）
- `fgets(name, sizeof name, stdin)` 可读取含空格的一整行；容量 20 时最多存 19 个输入字节和 `\0`，成功时返回非 `NULL`。（p30）
- `fgets` 在缓冲区有余量时会保留输入的换行 `\n`；长行可能要分多次读取。（p30）
- 安全删换行的惯用法是 `name[strcspn(name, "\n")] = '\0';`：有换行则定位并替换，无换行则定位到原有终止位置。（p31）
- `gets` 没有容量限制，已在 C11 被移除，绝不能使用。（p31）
- 手写 `my_strlen`：从 `n = 0` 开始，`while (s[n] != '\0') n++;`，再返回 `n`。无需长度参数，但每次调用要扫描，时间为 `O(n)`。（p32）

模块 3 | `<string.h>`：测量、复制、拼接、比较、搜索与编辑

3.1 工具分类与 `strlen` 的复杂度

- `<string.h>` 的六类工作：长度 `strlen`；复制 `strcpy` / `strncpy`；拼接 `strcat` / `strncat`；比较 `strcmp` / `strncmp`；搜索 `strchr` / `strrchr` / `strstr`；提取或编辑则通常用数组操作手写。C 没有标准 `substr`。（p35）
- `strlen(s)` 每次调用都从开头扫描到 `\0`，因此本身是 `O(n)`。（p36）

- 循环条件内反复写 `i < strlen(s)`，即使循环体是常数工作，也会进行约 `n + 1` 次完整扫描，总计 `O(n²)`。（p36，已按渲染公式核验）
- 正确模式：先写 `size_t n = strlen(s);`，随后 `for (size_t i = 0; i < n; i++)`；一次测长加一次遍历，总计 `O(n)`。（p36，已按渲染公式核验）

3.2 复制与拼接的容量契约

- `strcpy(dest, src)` 复制源串全部字符以及终止的 `\0`；不能用 `=` 复制数组内容。（p37）
- 使用 `strcpy` 的容量规则是 `capacity(dest) >= strlen(src) + 1`，且源与目标不能重叠。（p37）
- `strncpy(dest, src, sizeof dest - 1); dest[sizeof dest - 1] = '\0';` 可限制复制量，但 `strncpy` 自身可能不补终止符，因此补写是必要的。（p37）
- `snprintf(dest, sizeof dest, "%s", src)` 是较稳妥的复制替代：当容量大于 0 时会终止字符串；返回值表示本来需要写出的字符数，不表示输出到屏幕。（p37–p38）
- `strcat(s, suffix)` 先寻找 `s` 的原 `\0`，再从该处写入后缀并在末尾写新 `\0`；原终止符被覆盖。（p39）
- 拼接的容量规则：`capacity(s) >= strlen(s) + strlen(suffix) + 1`。（p39）

3.3 比较、搜索、截取与原地编辑

- `strcmp(a, b)` 按第一个不同字节比较：返回 `0` 表示相等，负值表示 `a` 字典序更小，正值表示更大；判断相等必须与 `0` 比较。（p40）
- 比较也会把 `\0` 纳入规则，例如 `"abc" > "ab"`，因为 `'c' > '\0'`。课件中的数值比较以 ASCII 为前提。（p40）
- `strchr(s, c)` 找字符首次出现位置；`strrchr(s, c)` 找最后一次出现；`strstr(s, t)` 找子串首次出现；找不到均返回 `NULL`。（p41）
- 这些搜索函数返回的是指向匹配位置的指针。若 `p = strchr(s, 'o')`，索引可用 `p - s` 计算；使用前必须确认 `p != NULL`。（p41）
- 手写子串提取时，目标数组需含额外终止位。例如从 `s + 1` 复制 3 个字节到 `char sub[4]` 后，必须 `sub[3] = '\0'`。（p42）
- 截断字符串可直接写 `s[index] = '\0'`；从头替换固定数量字符时，`strncpy(s, "Mon", 3)` 不会自动加 `\0`，原后缀会保留。（p43）
- 原地插入要求预留最终容量，并向右移动包括 `\0` 在内的尾部数据。（p43）

3.4 字符串数组、综合流程与缓冲区安全

- 二维字符数组可存“等宽槽位”的多条字符串：`char planets[4][8] = {...};` 每一行是一条字符串，第二维宽度必须至少为最长单词长度加 1。（p44）
- `planets[i][j]` 是第 `i` 条字符串的第 `j` 个字符；`printf("%s\n", planets[i])` 可逐行输出。（p44）
- 一个综合流程可依次执行 `strcpy`、`strcat`、`strlen`、字符下标访问、`strcmp` 与 `strstr`；字符串 `"Hello World"` 长度为 11，而 `"World"` 的首位置索引为 6。（p45）

- 核心函数风险表：`strlen` 不要重复计算；`strcpy` 与 `strcat` 要目标容量；`strncpy` 可能无 `\0`；`strcmp` 不能用 `==`；三种搜索函数均可能返回 `NULL`。（p46）
- **缓冲区溢出（buffer overflow）**是写入超过数组容量，例如 `char buf[8]` 不能容纳 `"Hello, World!"` 及其 `\0`；结果是未定义行为。（p47）
- 安全原则：始终为 `\0` 预留一个位置；限制输入宽度（如 `%19s`、`fgets`）；有界写入使用真实容量；写后确保终止符存在。（p47）
- 未定义行为下不能断言相邻变量一定被覆盖。课件展示“`score` 紧跟 `buf`”时可能改变 `score` 的假设场景，但 C 不保证内存布局，不能把该图当作确定结果。（p48）

模块 4 | `<ctype.h>` 与字符级字符串处理

4.1 ASCII 与字符和数值的互换

- 字符在 C 中是小整数。常用 ASCII：`'\0' = 0`、空格为 32、`'0' = 48`、`'9' = 57`、`'A' = 65`、`'Z' = 90`、`'a' = 97`、`'z' = 122`。（p50）
- 在 ASCII 下，`'a' - 'A' = 32`；数字字符转数值可用 `'7' - '0' = 7`。`printf("%d", 'A')` 输出 65。（p50）

4.2 安全的字符测试与大小写转换

- `<ctype.h>` 的测试函数：`isalpha` 字母、`isdigit` 数字、`isalnum` 字母或数字、`isupper` 大写、`islower` 小写、`isspace` 空白、`ispunct` 标点、`isxdigit` 十六进制数字、`isprint` 可打印字符。（p51）
- 这些测试函数返回非零表示真、0 表示假；参数只能是 `EOF` 或转换后的 `unsigned char` 值。（p51）
- 对 `char c` 的安全调用形式是 `isalpha((unsigned char)c)`；直接传入可能为负的普通 `char`（例如扩展字符）不符合函数契约。（p51）
- `EOF` 是 `<stdio.h>` 中定义的负 `int`，通常为 `-1`，由 `getchar()` 在输入结束时返回。（p51）
- `toupper` 与 `tolower` 分别转换英文字母大小写；非字母不变。真实程序用 `toupper((unsigned char)c)`，不要手写 ASCII 运算来代替库函数。（p52）

4.3 字符串验证与原地转换函数

- 仅数字字符串的判定：空串先返回 0；随后逐个字符以 `isdigit((unsigned char)s[i])` 检查，任一失败返回 0，全部通过返回 1。（p53）
- 该判定只验证“全部为数字字符”，不完成数值转换和范围检查；实际解析整数还需要范围验证。（p53）
- `const char s[]` 将输入承诺为只读；编译器会拒绝函数体对 `s[0]` 的写入，调用方可安全传字符串字面量。（p53）
- 原地大写函数用 `for (int i = 0; s[i] != '\0'; i++) s[i] = toupper((unsigned char)s[i]);`。数组按地址传递，因此函数无需返回值，调用方原字符串会被修改。（p54）

4.4 回文与双指针

- **回文（palindrome）**是正向与反向读相同的词、短语、数字或字符序列；可按题意忽略空格、标点和大小写。（p55）
- 基础解法先算 `n = strlen(s)`；空串直接返回 1；否则设置 `i = 0`、`j = n - 1`。（p55–p56）
- 当 `i < j` 时比较 `s[i]` 和 `s[j]`；若不同立即返回 0，相同则 `i++`、`j--`。两指针相遇或交错时返回 1。（p56）
- 双指针回文算法为 `O(n)` 时间、`O(1)` 额外空间。（p56）

复习自检｜原稿疑点与易错点

易错点	应记结论	来源
数组参数上的 `sizeof`	参数已经是指针，长度必须显式传入。	p16、p18
`strlen` 放在循环条件	会重复完整扫描，复杂度从 `O(n)` 恶化为 `O(n²)`。	p36
`strncpy`	复制被截断时可能没有 `\0`，需要显式终止。	p37
溢出图示	相邻变量被改写只是可能情形；未定义行为不保证具体内存结果。	p47–p48
`ctype.h` 参数	一律传 `EOF` 或 `(unsigned char)c`。	p51–p52

复习目标｜考前自检清单

> 逐条自问：合上笔记，能否说出或写出这一条？能就打勾。

1｜数组基础

- [ ] 能解释数组的同类型、连续、固定容量、零基下标四个性质，并写出合法下标范围。（p9）
- [ ] 能根据基地址与元素大小计算 `a[i]` 地址，并区分下标表达式和元素值表达式。（p10、p12）
- [ ] 能正确初始化数组，解释部分初始化、未初始化局部数组和 `LEN` 宏的限制。（p11、p16）
- [ ] 能写出不越界的遍历、求最大值和数组函数接口，并说明为什么参数还要有长度。（p14, p18）
- [ ] 能按行优先解释二维数组及其双层循环访问。（p19）

2｜C 字符串

- [ ] 能画出字符串的字节布局，明确 `\0`、`'0'` 与空字符串的区别。（p22, p23）
- [ ] 能用四种声明方式创建字符串，并为终止符留出容量。（p24）

- [ ] 能区分 `sizeof`、`strlen`、容量和长度，并对 `size_t` 使用 `%zu`。（p25）
- [ ] 能识别数组赋值、`==` 比较文本、修改字面量、缺少 `\0` 等错误。（p26）
- [ ] 能安全地使用 `%s`、`puts`、带宽度的 `scanf`、`fgets` 与 `strcspn` 删除换行。（p29, p31）

3 | `<string.h>`

- [ ] 能解释何时缓存 `strlen` 结果，并推导反复调用为何造成 `O(n²)`。（p36）
- [ ] 能为 `strcpy`、`snprintf`、`strcat` 写出目标容量条件及终止符要求。（p37, p39）
- [ ] 能依据 `strcmp` 的 0/负/正返回值判断文本关系，并拒绝使用 `==`。（p40）
- [ ] 能用 `strchr`、`strrchr`、`strstr` 搜索并处理 `NULL`，再用指针差求索引。（p41）
- [ ] 能实现固定长度截取、原地截断和字符串数组，并解释缓冲区溢出为何是未定义行为。（p42, p48）

4 | `<ctype.h>` 与练习函数

- [ ] 能使用 ASCII 的字符差完成基本字符/数字转换，并说明其前提。（p50）
- [ ] 能为所有 `ctype.h` 测试和转换函数正确传入 `(unsigned char)c`。（p51, p52）
- [ ] 能写出仅数字字符串验证与原地转大写函数，并正确使用 `const`。（p53, p54）
- [ ] 能用双指针写出回文判断，处理空串，并给出 `O(n)` 时间、`O(1)` 空间复杂度。（p55, p56）

自测专栏

自测 1｜查找方法（p6）

> **Quick check: one search on unsorted data. Which method fits?**
>
> 译：快速检查：对未排序数据执行一次查找，哪种方法适用？

> 答案（补充，非课件直接给出的答案，按 p6 规则推得）：顺序查找（sequential search），因为二分查找要求数组已排序。（p6）

自测 2｜数组循环输出（p13, p14）

> **What is stored? What is printed?**
>
> 译：存入数组的值是什么？输出结果是什么？
>
> `temp[m] = 10 + m * 2;`，随后从 `m = 4` 递减输出。
>
> 译：程序依公式写入数组，再将下标从 4 递减至 0 输出。

> 答案（p14）：`temp` 为 `{10, 12, 14, 16, 18}`；输出为 `18 16 14 12 10`。

自测 3｜字符串声明是否合法（p27, p28）

> **Valid string or not? Bonus: what is sizeof and strlen for each valid one?**
>
> 译：下列是否是有效字符串？加分题：每个有效项的 `sizeof` 和 `strlen` 是多少？
>
> `char a[] = "cat";`
>
> `char b[3] = "cat";`
>
> `char c[4]; c = "cat";`
>
> `char d[8] = "cat";`
>
> 译：以上依次为自动推断容量、容量恰为字符数、数组声明后整体赋值、以及留有额外空间的四种声明。

> 答案（p28）：`a` 有效，大小 4、长度 3；`b` 没有 `\0`，不是字符串；`c = "cat"` 是数组不可赋值的编译错误，应使用 `strcpy(c, "cat")`；`d` 有效，大小 8、长度 3。

自测 4｜字面量的大小（p33）

> **printf(“%zu”, sizeof(“Hello”)); // prints ?**
>
> 译：`sizeof("Hello")` 输出什么?

> 答案（补充，非课件直接给出的答案，按 p23 推得）：`6`，因为字符串字面量含五个可见字符和一个隐藏 `\0`。（p23、p33）

自测 5｜输入字符串的存储差异（p30, p31）

> **Try separately: type Big Cat, then press Enter.**
>
> 译：分别运行：输入 `Big Cat`，然后按 Enter。

> 答案（p30–p31）：`scanf("%19s", name)` 存为 `"Big\0"`；`fgets(name, 20, stdin)` 存为 `"Big Cat\n\0"`。随后可将首个 `\n` 改为 `\0`。
>
> 译：答案中函数名是代码标识符；其含义是单词读取会在空格停止，而整行读取会保留换行。

自测 6｜原地插入（p43）

> **Lab: insert “and” into “begin end”. The result needs 14 bytes.**
>
> 译：实验：将 `"and "` 插入 `"begin end"`。结果需要 14 个字节。

> 答案：unanswered（未作答）。课件未给出具体实现；应预留容量，并从尾部开始右移（包括 `\0`），再复制 `"and "`。（p43）
>
> 译：unanswered 表示课件没有提供答案。

自测 7｜回文判断（p55, p56）

> **Write int is_palindrome(const char s[])**
> **Challenge: Make “Never odd or even” pass: skip spaces, ignore case.**
>
> 译：编写 `int is_palindrome(const char s[])`。挑战：让 `"Never odd or even"` 也通过，要求跳过空格并忽略大小写。

> 答案（基础题，p56）：空串返回 1；`i = 0`、`j = strlen(s) - 1`，在 `i < j` 时比较两端字符，不同返回 0，相同则向中间移动，结束返回 1。

> 挑战答案：unanswered（未作答）。课件只给出要求，没有给出实现；实现时可用 `isspace` 跳过空白，并以 `tolower((unsigned char)c)` 进行无大小写比较。（p51、p52、p55）
>
> 译：unanswered 表示课件没有提供答案。